Prompt Documentation

Prompt Documentation

Purpose Of This Document

This document records the prompts used for knowledge-engineering tasks in the current repository snapshot, together with the task each prompt supports. The coursework brief asks for prompts to be documented explicitly, so this file distinguishes between:

- prompts preserved exactly in the repository
- prompt-driven tasks that can only be reconstructed from surrounding code and artifacts

The current pipeline keeps one extraction prompt verbatim in code and preserves clear evidence of several additional prompt-assisted activities around source selection, ontology alignment, and competency-question development.

Exact Prompt Recoverable From The Current Repository

Task: triple extraction from unstructured text

Source: `data_retrieval/unstructured_retrieval/extract_triples.py`

Current runtime configuration recorded in code:

- endpoint: `https://api.anthropic.com/v1/messages`
- authentication variable: `API_KEY`
- model variable: `model`
- current repo mode: `USE_LLM = False`
- current cache file: `data/caches/triples_cache.json`

In the committed repository, the unstructured stage is configured for cache-backed reproducibility. The exact prompt is still preserved in code, and a live run can be enabled locally by supplying the desired credentials and model configuration.

Repository-preserved prompt:

```
You extract RDF triples from text about London public transport.

            Return ONLY lines as: subject | predicate | object
            No bullets, numbers, markdown, or explanation. If nothing to extract: NONE

            ALLOWED PREDICATES (use only these):
            operatedBy, hasStop, hasTerminus, servedByLine, connectsTo,
            operatesInZone, isNightService, isNightTube, hasStepFreeAccess,
            hasAccessibilityFeature, acceptedOn, openedIn, isFlatFare,
            routeNumber, terminatesAt, locatedIn, partOf, replacedBy,
            introducedBy, hasFacility
```

```
RULES:
- Only facts explicitly in the text. Never infer.
- Short labels without articles: "Victoria line" not "the Victoria line"
- Boolean values as "true"
- Years as 4 digits
- Zones as "Zone 1", "Zone 2"
- Max 12 triples
- Ensure data is correct as of 2026

EXAMPLES:
Input: "The Victoria line was opened in 1968 and runs from Brixton to Walthamsto
Output:
Victoria line | openedIn | 1968
Victoria line | hasTerminus | Brixton
Victoria line | hasTerminus | Walthamstow Central
Victoria line | operatedBy | London Underground

Input: "Stratford station is served by the Jubilee line, Central line, and DLR.
Output:
Stratford station | servedByLine | Jubilee line
Stratford station | servedByLine | Central line
Stratford station | servedByLine | DLR
Stratford station | operatesInZone | Zone 3

Input: "London Buses operate a flat fare of £1.75. The Oyster card and contactle
Output:
London Buses | isFlatFare | true
Oyster card | acceptedOn | London Buses
Contactless payment | acceptedOn | London Buses
```

Knowledge-engineering task supported:

- extracting candidate RDF triples from textual sources
- constraining the predicate vocabulary used by the extractor
- shaping extracted facts toward the ontology and competency-question themes

Why this prompt is effective:

- it sharply limits the allowed predicate set
- it encourages concise entity labels
- it prevents explanatory text from leaking into the extraction output
- it includes domain-specific examples covering lines, stations, zones, and fares

Prompt-adjacent control logic

The current extractor also relies on non-prompt controls that shape prompt output before RDF insertion:

- `chunk_text(...)` limits passages to roughly 1,000 characters
- `filter_triples_by_entities(...)` retains triples that match recognized entities
- `remove_junk(...)` filters malformed or low-value extractions

These functions are not prompts themselves, but they form part of the overall prompt-engineering strategy because they constrain how model outputs are accepted into the

knowledge graph.

Cache And Reproducibility Notes

The current repository is intentionally reproducible in cache-backed mode:

- `USE_LLM = False`
- the extraction cache currently contains 1,442 entries

This is helpful in a coursework setting because it preserves a stable graph-generation path even when live external services are not being called. The exact request-by-request history that produced the cache is not fully recoverable from the repository alone, but the prompt template, surrounding code, and cached outputs are all preserved.

Prompt-Driven Activities Evidenced Indirectly

Task: predicate-to-ontology mapping refinement

Source evidence:

- `data_retrieval/unstructured_retrieval/triples_to_rdf.py`
- comments indicating prompt-assisted support for predicate mapping

What is preserved:

- the final predicate mapping used during RDF construction

What is not preserved:

- the exact original prompt text used to build or refine that mapping

Repository-consistent reconstructed prompt:

```
Given the following raw predicate strings extracted from London public transport text, map e
```

Knowledge-engineering task supported:

- aligning extracted surface predicates with the ontology property layer

Task: source selection for the expanded Wikipedia corpus

Source evidence:

- `data_retrieval/unstructured_retrieval/wiki_scraper.py`
- the curated list of transport-related article titles in the repository

What is preserved:

- the final selected article list

What is not preserved:

- the exact prompts or search strings used to arrive at that list

Repository-consistent reconstructed prompt:

```
Identify Wikipedia pages that are likely to provide explicit facts for a London transport kn
```

Knowledge-engineering task supported:

- expanding textual source coverage
- aligning source selection with competency-question themes

Task: competency-question augmentation

Source evidence:

- the coursework brief requires 10 manual and 10 LLM-augmented competency questions
- the repository preserves the final question set but not the original dialogue that produced it

Repository-consistent reconstructed prompt:

```
Generate additional competency questions for a London public transport knowledge graph that
```

Knowledge-engineering task supported:

- extending the requirements set beyond the manually written core questions
- improving thematic coverage of the final SPARQL query suite

Assessment Of The Prompt Layer

The current prompt layer shows a thoughtful move toward constrained extraction:

1. the extraction prompt uses a restricted predicate vocabulary
2. the examples are domain-specific and relevant to the ontology
3. the prompt is embedded in a cache-backed and filter-assisted processing pipeline

The main opportunities for further strengthening are:

1. tighter entity typing before RDF materialization
2. stronger validation between extracted entities and canonical ontology classes
3. continued synchronization between prompt-driven source selection and benchmark query needs

Conclusion

The repository preserves the current extraction prompt exactly and also gives enough surrounding evidence to document other prompt-assisted tasks in the pipeline. Taken together, these prompts support source discovery, competency-question augmentation, predicate mapping, and textual triple extraction, which are all central knowledge-engineering activities in the coursework brief.